

INTERNSHIP PROJECT REPORT

Secure Notes App

A local demonstration of encrypted note storage and controlled decryption

Submitted by	Pothula Yashwanth
Email	pothulayashwanth55@gmail.com
College	Presidency University
Semester	5th
Internship ID	HH8-9014638
Submission Date	26 July 2026

Prepared for internship evaluation

This report documents a local educational encryption demonstration. The contents shown are sample notes only.

1. Introduction

The Secure Notes App demonstrates how sensitive notes can be protected before being stored on a computer. Instead of saving readable text, the application encrypts each note and writes only encoded encrypted data to a local JSON file. The note can be recovered only when the correct passphrase is supplied.

2. Objective

The objective is to demonstrate secure local storage using authenticated encryption. The project shows the complete process of encrypting sample notes, storing ciphertext with its required parameters, decrypting with the correct passphrase, and safely rejecting an incorrect passphrase.

3. Technologies Used

- **Python:** Implements the application workflow, file handling, and encryption operations.
- **Cryptography library:** Provides modern encryption primitives used to protect the note data.
- **AES-GCM:** Supplies confidential storage plus authentication, so incorrect or altered data cannot be read as valid notes.
- **PBKDF2 key derivation:** Derives an encryption key from the user passphrase and a unique salt.
- **JSON:** Stores the non-secret encryption parameters and ciphertext in a portable local file.

4. Working Principle

For each note, the application derives an AES key from the passphrase using PBKDF2 and a salt. It then generates a nonce and encrypts the note with AES-GCM. The JSON file stores the salt, nonce, and ciphertext; the readable note and the derived key are not written to disk. When the correct passphrase is entered later, the same derivation process creates the required key and the note is decrypted.

Execution Flow

- Accept a sample note and passphrase in the local demonstration.
- Generate or use a salt, then derive an AES key with PBKDF2.
- Encrypt the note using AES-GCM and a nonce.
- Save only salt, nonce, and ciphertext to `secure_notes.json`.
- Derive the key again from the correct passphrase to decrypt the note.
- Reject the decryption request when the incorrect passphrase is used.

5. Detailed Explanation

Salt: A salt is non-secret random data used during key derivation. It ensures that the same passphrase can result in a different derived key when a different salt is used.

Nonce: The nonce is a value required by AES-GCM for a single encryption operation. It is stored so the encrypted data can be correctly processed later.

Ciphertext: Ciphertext is the unreadable encrypted form of the original note. It is safe to store in the JSON file because it cannot be converted back into readable text without the correct key.

Authentication: AES-GCM verifies integrity during decryption. A wrong passphrase produces the wrong key, so the program fails safely instead of returning a false result.

6. Features

- Encrypts notes before writing them to disk.
- Stores only ciphertext and encryption parameters in a JSON file.
- Uses a passphrase-derived encryption key rather than saving the key in the file.
- Decrypts notes correctly only with the matching passphrase.
- Safely reports failure when a wrong passphrase is used.
- Demonstrates clearing the derived key from memory after use.

7. Advantages

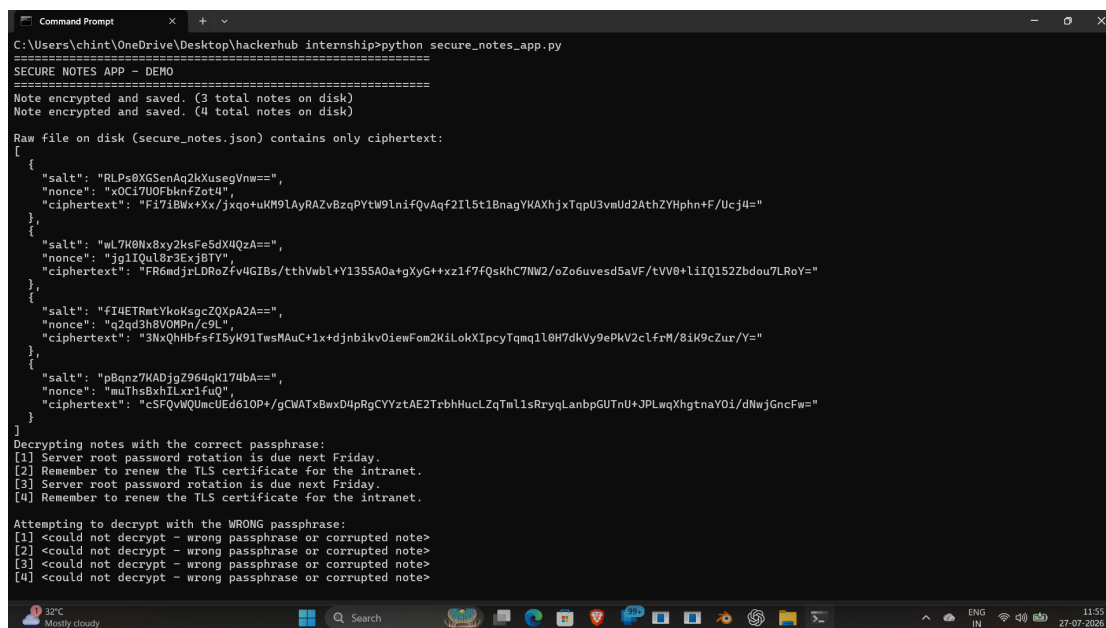
- Keeps sensitive note text from appearing directly in the storage file.
- Provides confidentiality and integrity with authenticated AES-GCM encryption.
- Uses a portable JSON representation that is simple to inspect for educational purposes.
- Demonstrates essential secure-storage concepts in a compact Python project.

8. Applications

- Personal encrypted note storage demonstrations.
- Cybersecurity and cryptography laboratory exercises.
- Examples for teaching key derivation, encryption, and secure file handling.
- A foundation for extending into a password-protected local notes tool.

9. Implementation Evidence

The screenshots below show the full demonstration: successful encryption and storage of notes, the ciphertext-only JSON data, correct-passphrase decryption, and safe rejection of an incorrect passphrase.



```
Command Prompt
C:\Users\chint\OneDrive\Desktop\hackerhub_internship>python secure_notes_app.py

=====
SECURE NOTES APP - DEMO
=====
Note encrypted and saved. (3 total notes on disk)
Note encrypted and saved. (4 total notes on disk)

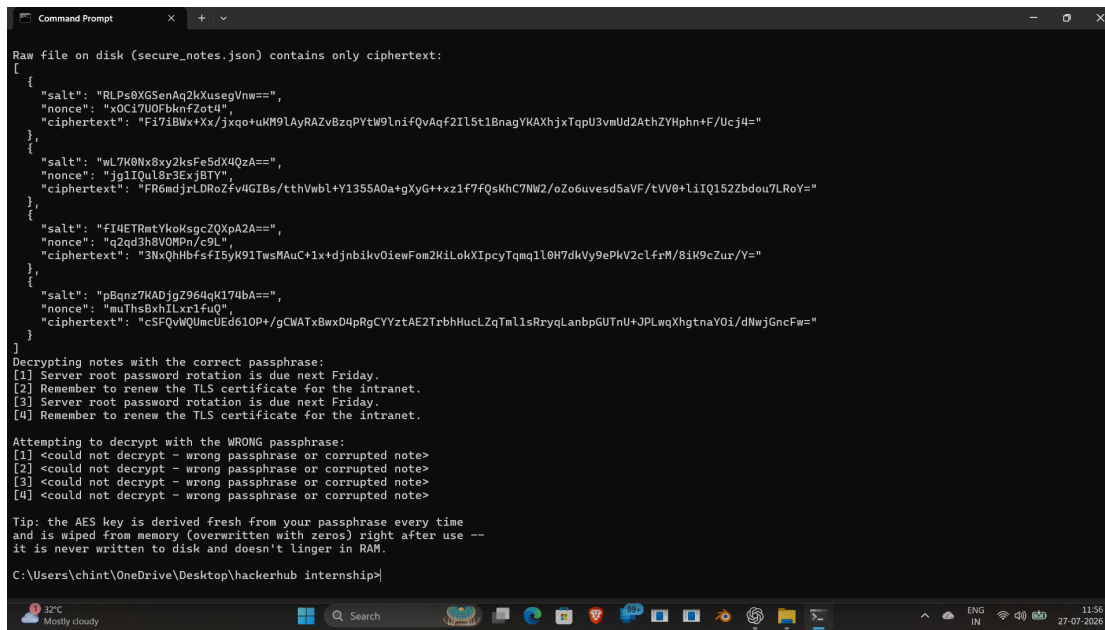
Raw file on disk (secure_notes.json) contains only ciphertext:
[
  {
    "salt": "RLPs0XGSeNq2kXusegVnw==",
    "nonce": "x0C17U0fbknfZot4",
    "ciphertext": "F17iBWx+Xx/jxqo+uWM9lAyRAZvBzqPYtW9lnifQvAqf2IL5t1BnagYKAXhjxTqpU3vmUd2AthZYHphn+F/Ucj4="
  },
  {
    "salt": "wL7K0N8xy2ksFeSdX4QzA==",
    "nonce": "jglIQu18r3ExJBTY",
    "ciphertext": "FR6mdjrLDroZfv4GIBs/tthVwbl+Y1355A0a+gXyG+xxz1f7fQsKhC7NMW2/oZo6uveds5aVF/tVV0+LiIQ152Zbdou7LRoY="
  },
  {
    "salt": "fI4ETRmtYkoKsgcZQXpA2A==",
    "nonce": "q2qd3h8VOMpN/c9L",
    "ciphertext": "3NxQhHbfsfI5yK91TwsMAuC+lx+djnbikv0iewFom2KiLokXIpcyTmqm1l0H7dkVy9ePkV2clfrM/8iK9cZur/Y="
  },
  {
    "salt": "pBqnz7KADjgZ964qk174bA==",
    "nonce": "muThsBxhILxr1fuQ",
    "ciphertext": "cSFQvWQumcUEd610P+/gCWATx8wxD4pRgCYZtAE2TrbhHuclZqTml1sRryqLanbpGUTnU+JPLwqXhgtnaYOi/dNwjGncFw="
  }
]

Decrypting notes with the correct passphrase:
[1] Server root password rotation is due next Friday.
[2] Remember to renew the TLS certificate for the intranet.
[3] Server root password rotation is due next Friday.
[4] Remember to renew the TLS certificate for the intranet.

Attempting to decrypt with the WRONG passphrase:
[1] <could not decrypt - wrong passphrase or corrupted note>
[2] <could not decrypt - wrong passphrase or corrupted note>
[3] <could not decrypt - wrong passphrase or corrupted note>
[4] <could not decrypt - wrong passphrase or corrupted note>
```

Figure 1. The Secure Notes App encrypts sample notes, displays ciphertext-only JSON data, decrypts with the correct passphrase, and rejects a wrong passphrase.

9. Implementation Evidence (continued)



```
Command Prompt
Raw file on disk (secure_notes.json) contains only ciphertext:
[
  {
    "salt": "RLPe0YGSenAq2LXusegVnmw==",
    "nonce": "x0C17U0fBbnfZot4",
    "ciphertext": "Fi7iBWx+Xx/jxqo+uKM9LAyRAZvBzqPYtW9LnifQvAqf2IL5t1BnagYKAXhJxTqU3vmUd2AthZYHphn+F/Ucj4="
  },
  {
    "salt": "wL7K0Nx8xy2ksFeSdX4QzA==",
    "nonce": "jglIQUl8r3ExJBTY",
    "ciphertext": "FR6mdjrlDRoZfv4GIBs/tthVwbl+Y1355A0a+gXyG++xz1f7fQsKhC7NW2/oZo6uvesd5aVF/tVV0+LiIQ152Zbdou7LRoY="
  },
  {
    "salt": "fi4ETRmtYkoKsgcZQXpA2A==",
    "nonce": "q2qd3h8VOMpN/c9L",
    "ciphertext": "3NxQhHbfsfI5yK91TwsMAuC+1x+djnb1kV0iewFom2KiLokXIpcyTmq1l0H7dkVy9ePkV2clfrM/8iK9cZur/Y="
  },
  {
    "salt": "pBqnz7KADjgZ964qK174bA==",
    "nonce": "muThsBxhILxrIfuQ",
    "ciphertext": "cSFQvQUMcUEd610P+/gCWATxBwxD4pRgCYZtAE2TrbhHucLZqTml1sRryqLanbpGutNU+JPLwqXhgtNaYoi/dNwjGncFw="
  }
]
Decrypting notes with the correct passphrase:
[1] Server root password rotation is due next Friday.
[2] Remember to renew the TLS certificate for the intranet.
[3] Server root password rotation is due next Friday.
[4] Remember to renew the TLS certificate for the intranet.

Attempting to decrypt with the WRONG passphrase:
[1] <could not decrypt - wrong passphrase or corrupted note>
[2] <could not decrypt - wrong passphrase or corrupted note>
[3] <could not decrypt - wrong passphrase or corrupted note>
[4] <could not decrypt - wrong passphrase or corrupted note>

Tip: the AES key is derived fresh from your passphrase every time
and is wiped from memory (overwritten with zeros) right after use --
it is never written to disk and doesn't linger in RAM.

C:\Users\chint\OneDrive\Desktop\hackerhub internship>
```

Figure 2. The local JSON data contains encryption parameters and ciphertext; correct-passphrase decryption succeeds, while all wrong-passphrase attempts fail safely.

10. Conclusion

The Secure Notes App was successfully demonstrated in a local environment. It encrypts notes before storage, retains only salt, nonce, and ciphertext in the JSON file, and restores the readable text only with the correct passphrase. The incorrect-passphrase test confirms that the application fails safely. This project provides a clear practical demonstration of AES-GCM, PBKDF2 key derivation, and secure local data handling.